

Chair of Computer Architecture (M. Sc.)

Model Enumeration using IPASIR-UP without Blocking Clauses

Master's Thesis

First Examiner

Prof. Dr. Biere

Second Examiner

Prof. Dr. Scholl

Author

Timpe Horig

Matrikelnummer: 5108196

Geburtsort: Titisee-Neustadt, Germany

Working Period

18.11.2025 – 18.05.2026

DECLARATION

I hereby declare, that I am the sole author and composer of my thesis and that no other sources or learning aids, other than those listed, have been used. Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.

I also hereby declare that my thesis has not been prepared for another examination or assignment, either in its entirety or excerpts thereof.

Freiburg im Br. 18.05.2026

Place, date


Signature

Abstract

The Boolean satisfiability problem (SAT) is a fundamental NP-complete problem with numerous applications in verification and reasoning. While modern Conflict-Driven Clause Learning (CDCL) based SAT solvers efficiently compute single satisfying assignments, many applications require enumerating all solutions (AllSAT). This is challenging due to their potentially exponential number and the inefficiency of standard techniques such as blocking clauses.

Implementing a model enumerator through the IPASIR-UP interface allows to leverage highly optimized SAT solvers such as CaDiCaL while maintaining a clear separation between the solver and the enumeration logic. This modular approach ensures that the enumerator remains independent of a specific solver implementation, requiring only compliance with the interface. As a result, improvements in underlying solvers can be directly utilized without modifying the enumerator, enabling both flexibility and long-term maintainability.

This thesis presents CaDiCaL: a model enumerator without blocking clauses using the IPASIR-UP interface.

Acknowledgements

I would like to express my sincere gratitude to my supervisor Florian Pollitt for the continuous support, valuable feedback, and insightful discussions throughout the course of this thesis.

I am also deeply grateful to my family and my girlfriend for their unwavering support and encouragement during my studies.

Contents

1	Introduction	1
2	IPASIR	3
3	IPASIR-UP	4
4	CaDiCall	6
4.1	Conflict-Driven Clause Learning	6
4.2	Chronological Backtracking	7
4.2.1	Backtracking After a Model Is Found	8
4.2.2	Conflict Induced Backtracking	10
4.3	Implicant Shrinking	11
4.4	Deciding	14
4.5	Options	15
5	Extending IPASIR-UP	17
5.1	Necessary Extensions for Model Enumeration	17
5.1.1	Backtracking After a Model Is Found	17
5.1.2	Maintaining Chronological Backtracking	18
5.1.3	Out Of Order Propagations	19
5.2	Implicant Shrinking	19
5.3	Optimization	20
5.3.1	Interacting With the Solver	21
5.3.2	Duplicating Datastructures	21
5.3.3	Storing Models	21
6	Experiments	22
6.1	Configuration	22
6.2	Baseline	23
6.3	Benchmark Sets	23
6.4	Evaluation Metrics	24
6.5	Results	24
6.5.1	Benchmark Set from TabularAllSat	27
6.5.2	Benchmark Set from SATLIB	31
6.5.3	Benchmark Set from the Model Counting Competition 2025	32
6.6	Mismatching Results	34
6.7	Benchmark Results Overview	35
7	Furture Work	36
8	Conclusion	37

1 Introduction

The Boolean satisfiability problem (SAT) is defined as follows [5]:

Given a propositional formula F over variables $V = \{v_1, \dots, v_n\}$, does there exist an assignment $\alpha : V \rightarrow \{0, 1\}$ such that $\alpha \models F$.

This problem was the first to be proven NP-complete by Cook [9] in 1971 and independently by Levin [13] in 1973. As a result, finding efficient algorithms for the SAT problem quickly became one of the most important and influential challenges in computer science [3]. Modern Conflict-Driven Clause Learning (CDCL) solvers are capable of handling formulas with millions of variables and clauses [8]. As a result, SAT solving plays a central role in hardware and software verification, automated reasoning, planning, scheduling, model checking, and cryptanalysis [5] [14] [16].

State-of-the-art solvers such as CaDiCaL [6], Glucose [2], and Kissat [7] demonstrate how theoretical advances combined with low level optimizations can transform an intractable worst-case problem into a powerful practical tool. These solvers typically focus on the decision problem: Determining whether at least one satisfying assignment exists. If the formula is satisfiable, the solver returns a single model.

However, many applications require more than just one solution. In areas such as configuration, diagnosis, quantitative reasoning, explanation generation, and formal verification, it is often necessary to enumerate all satisfying assignments [1] [4] [5] [11]. This task, known as model enumeration (or AllSAT), generalizes the classical SAT problem from computing a single satisfying assignment to computing the whole set $\{\alpha : V \rightarrow 0, 1 \mid \alpha \models F\}$. While conceptually straightforward, model enumeration introduces significant algorithmic challenges. The number of satisfying assignments may be exponential in the number of variables, and naive approaches can quickly become infeasible [15].

A common technique for model enumeration is the iterative use of SAT solvers combined with blocking clauses. After each model is found, a clause composed of the negation of every literal in that model is added to the CNF formula, ensuring that the solver does not rediscover the same assignment.

With the introduction of standardized interfaces such as IPASIR [3], SAT solvers can be seamlessly integrated into larger systems without the need to maintain the solver itself. More recently, IPASIR-UP [10] was introduced, expanding IPASIR with user propagators. This allows adding clauses without restarting the search. Using IPASIR-UP with Blocking Clauses yields a very clean model enumerator that benefits from the underlying solver CaDiCaL while CaDiCaL itself remains unchanged. Although effective, this approach has several drawbacks: The repeated addition of blocking clauses increases the size of the formula, thereby degrading solver performance by affecting propagation efficiency and memory consumption. As the number of models grows, these effects scale up, eventually making blocking clauses infeasible.

In 2024 Spallitta et al. [15] presented TabularAllSat: A SAT solver that is especially designed for disjoint projected enumeration for SAT and SMT without blocking

clauses. They claim that TabularAllSat outperforms all other state-of-the-art AllSAT solvers. This was achieved by "combining the best of current AllSAT state-of-the-art literature": Conflict-Driven Clause Learning short CDCL, chronological backtracking to avoid blocking clauses and implicant shrinking to minimize models.

Besides adding clauses iteratively, IPASIR-UP also allows interfering with decision-making, backtracking and found models. This raises the question whether it is possible to create a model enumerator through IPASIR-UP without blocking clauses by using all state-of-the-art techniques from TabularAllSat.

This thesis introduces CaDiCaL: a model enumerator that showcases that model enumeration through the IPASIR-UP interface without blocking clauses is possible. However, CaDiCaL requires changes to both, the IPASIR-UP interface and CaDiCaL itself. Some of these changes are already added to the newest version of the IPASIR-UP interface, while others remain to be discussed. CaDiCaL implements all three state-of-the-art techniques defined by Spallitta et al. [15]: Conflict-Driven Clause Learning, chronological backtracking and implicant shrinking.

2 IPASIR

The IPASIR interface [3] standardizes incremental interaction with CDCL-based SAT solvers by defining a minimal set of functions through which the user can call the solver in bigger contexts while keeping the program independent of the solver. A solver instance is created and subsequently manipulated only via the following functions: The function `ipasir_add` to add literals to the current clause, terminated by zero. The function `ipasir_assume` to add temporary assumption literals and `ipasir_solve` to start or resume the search. After `ipasir_solve` returns SAT or UNSAT, the user may query models with `ipasir_val` or inspect a failed core via `ipasir_failed`. The interface deliberately abstracts from internal solver mechanisms: There is no access to decision levels, propagation events, learned clauses, or backtracking. Although learned clauses are preserved across calls in incremental mode, the user cannot intervene during the search itself. Consequently, model enumeration requires blocking clauses: After finding a model, add its negation as a new clause to the CNF (via `ipasir_add`) and restart the solver. If the solver returns UNSAT, all models are found. However, this approach is suboptimal as each new blocking clause increases the CNF. This makes it infeasible for large CNFs with many models. Model enumeration without blocking clauses through IPASIR is impossible since the solver terminates immediately upon finding a satisfying assignment.

3 IPASIR-UP

To overcome the limitations from IPASIR, Fazekas et al. [10] introduced IPASIR-UP in 2023 as an extension of IPASIR that introduces user propagators (UP). IPASIR-UP augments the classical interface with a bidirectional communication layer between the solver’s CDCL loop and user-defined callback functions. Instead of restricting interaction to clause addition and top-level solve calls, IPASIR-UP enables controlled intervention during propagation, decision making, backtracking, and model detection.

- Model validation

```
bool cb_check_found_model(const std::vector<int> &model) override {  
    return true;  
}
```

This function is called whenever the solver derives a total satisfying assignment. If the user propagator returns `true`, the solver reports SAT as usual. If it returns `false`, the model is rejected and the solver continues the search without restarting. Crucially, the internal trail, learned clauses, and heuristic state remain intact.

- External clause injection

```
bool cb_has_external_clause(bool &is_forgettable) override {  
    return false;  
}  
  
int cb_add_external_clause_lit() override { return 0; }
```

These functions allow the user propagator to provide clauses to the solver right after `cb_check_found_model` was called. When `cb_has_external_clause` returns `true`, the solver repeatedly calls `cb_add_external_clause_lit` to obtain the literals of the clause (terminated by zero).

- Notifications

```
void notify_assignment(const std::vector<int> &list) override {}  
void notify_new_decision_level() override {}  
void notify_backtrack(size_t new_level) override {}
```

These notification callbacks inform the user propagator about changes to the assignment trail. The callback function `notify_assignment` communicates newly assigned literals, `notify_new_decision_level` signals that a decision was made and thus the decision level increased, and `notify_backtrack` reports to which level the solver backtracked.

- Decision-making

```
int cb_decide() override { return 0; }
```

This callback is invoked before the solver makes the next decision. If it returns a non-zero literal, the solver uses it as the next decision. If zero is returned, the solver makes its own decision, based on its internal decision heuristic.

- Propagation

```
int cb_propagate() override { return 0; }  
int cb_add_reason_clause_lit(int propagated_lit) override {  
    return 0;  
}
```

`cb_propagate` enables the user propagator to return implied literals under the current partial assignment. If a conflict is detected in which the propagated literal was involved, the solver asks for a reason clause through repeatedly calling `cb_add_reason_clause_lit` (terminated by zero). This enables steering through the decision space while adding the reason clauses only if necessary.

A user must define a user propagator implementing all these functions above. Then the user propagator can be controlled by the following functions:

```
void connect_external_propagator (ExternalPropagator* propagator);  
void disconnect_external_propagator ();  
void add_observed_var (int var);  
void remove_observed_var (int var);
```

By calling `connect_external_propagator` and `disconnect_external_propagator`, the user can connect or disconnect the user propagator from the solver. The notification callback functions described above are only called by the solver for variables that are marked as `observed`. The user can mark and unmark variables through the functions `add_observed_var` and `remove_observed_var`.

4 CaDiCAll

In order to discover all models for one CNF, a model enumerator must continue exploring the search space after it found one model. A central challenge in this process is to avoid rediscovering previously found models. In the absence of blocking clauses, this can be achieved by enforcing a systematic traversal of the search space. Such an approach ensures that each region of the search space is explored exactly once, thereby guaranteeing that no model is generated more than once or omitted.

However, this approach introduces several challenges: First, modern SAT solvers are generally not designed to explore the search space in a strictly systematic manner, as they rely on heuristic-driven decision strategies and other features such as restarts or back-jumping. Second, since the search space can grow exponentially with the number of variables, a systematic exploration of the search space may become computationally infeasible.

Our model enumerator CaDiCAll which uses CaDiCaL almost exclusively through the IPASIR-UP interface overcomes these challenges by implementing all three state-of-the-art mechanisms for AllSAT solvers defined by Spallitta et al. [15]: Conflict-Driven Clause Learning (Section 4.1), chronological backtracking (Section 4.2) and implicant shrinking (Section 4.3).

4.1 Conflict-Driven Clause Learning

Conflict-Driven Clause Learning (CDCL) improves the classical DPLL algorithm by learning from conflicts. When the solver runs into a conflict, it does not simply flip the last decision but analyses the conflict to determine the assignments that caused the contradiction.

The analysis starts with the conflicting clause (the clause that is falsified by the current assignment). The solver inspects which literals in that conflict clause were forced by another clause and resolves them with each other. Let the conflict clause be $(A \vee \neg x_1)$ and x_1 be forced by $(B \vee x_1)$. Resolving these two clauses on x_1 yields the reason clause $(A \vee B)$. This process is repeated until a learned clause of the form $(A \vee B \vee \dots)$ is obtained that contains exactly one literal from the current decision level. This clause is then added to the CNF. Such a clause enables non-chronological backtracking: The solver backtracks to the second-highest decision level occurring in the clause. At this point, all other literals in the clause remain falsified while the literal from the previous decision level becomes the only unassigned one, making the clause unit and forcing a new assignment through unit propagation. This prevents the solver from encountering the same conflict again.

By learning new clauses and pruning conflicting regions of the search space, CDCL significantly improves the efficiency of SAT solving. For this reason, it forms the basis of most modern SAT solvers.

This advantage is inherited by CaDiCaL, as it uses the CDCL-based SAT solver CaDiCaL through the IPASIR-UP interface.

4.2 Chronological Backtracking

Model enumeration must ensure that each model is counted exactly once. This can be achieved by excluding paths that have already led to a model, from further exploration. In common model enumerators this is done by blocking clauses, which permit the solver to run into the same model again. Without blocking clauses we must ensure manually that every path is explored exactly once.

Modern SAT solvers based on Conflict-Driven Clause Learning (CDCL) employ non-chronological backtracking (NCB). During conflict analysis, the solver may determine that some earlier decisions did not contribute to the conflict and can therefore backtrack across multiple decision levels at once. This approach significantly improves efficiency by skipping parts of the search tree that cannot lead to a solution. However, it also changes the order in which decisions are revisited and effectively reshapes the search tree during the search. Therefore, NCB can introduce redundant work. In standard SAT solving, this is usually not an issue, since the solver stops after finding a single model, encountering the same model again is not a concern. However, when enumerating all models, the solver does not terminate after finding the first model. Instead, it must check the remaining search space for others. As NCB can alter the search space and does not uncover the search space systematically, the only possibility to prevent the solver from rediscovering the same model again is to exclude the exact combination of literals in this model, implying blocking clauses.

Exploring the search space systematically makes blocking clauses redundant. The idea is simple: When backtracking, always only backtrack to the last decision and flip it. If there is no more decision to flip, we can terminate as the whole search space is uncovered. This is known as chronological backtracking (CB). With CB all currently made decisions split the search space into two parts: The part that already has been uncovered and the part that is left to explore. With this approach it suffices to keep track of the current decisions instead of all models that have already been explored.

Transferring this idea to CaDiCaL imposes different challenges: First, backtracking after a model is found was not supported by the IPASIR-UP interface. Second, even though chronological backtracking can be set as an option in CaDiCaL, it does not guarantee to assign the negation of the old decision.

As we will discuss in Section 5, backtracking after a model is found is fairly easy to add and does not introduce any bad behavior. Thus, allowing to force CaDiCaL to backtrack when a model is found was added to the newest version of CaDiCaL’s IPASIR-UP interface by Katalin Fazekas.

To solve the latter issue, we could store the decisions and then assign them manually. As we will discuss in Section 4.4, the decision order as well as the phase of decisions can be

chosen freely and is left for CaDiCaL to decide. Therefore, we do not store the decision itself but for each decision level we store a counter (decision counter) and the decided variable. The decision counter indicates how often a variable was assigned. A count of one stands for a new decision while a count of two indicates that this decision was already flipped. The decision level CaDiCaL backtracks to then simply is the highest decision level with a decision count of one. After a backtracking occurred, CaDiCaL may retain certain literals from higher decision levels and propagate before CaDiCaL can continue its search. However, this may lead to different solver states. Let the previously made decision be denoted by d . In this context, four cases can be distinguished:

- **The solver propagates the previous decision d .** In this case, the solver re-enters the same search space. This is directly detected in `notify_assignment` as CaDiCaL notices that the previous decision has been assigned again. However, the earliest point at which this situation can be handled is either `cb_decide` or `cb_check_found_model`, since these are the only callback functions capable of enforcing backtracking. The function `notify_assignment` marks the current search path as a duplicate, and as soon as either `cb_decide` or `cb_check_found_model` is invoked, the solver is forced to backtrack further. This behavior is sound because all assignments leading to the propagation of d are themselves consequences of earlier decisions. Hence, the propagation of d is a direct implication of the current decision level.
- **The solver encounters a conflict during propagation.** In this case, conflict-induced backtracking is triggered. This mechanism is described in Section 4.2.2.
- **The solver propagates only literals different from d , or performs no propagation at all.**
- **The solver propagates $\neg d$.**

The first two cases are always the same while the handling of the latter two depends on the context in which a backtracking occurs and must be considered in the following sections.

4.2.1 Backtracking After a Model Is Found

When a model is found, CaDiCaL determines the highest decision level $d1$ with a decision count of one and then forces CaDiCaL to backtrack to the decision level $d1 - 1$. If this level is negative there exists no decision that has not been already flipped and the solver can terminate as there is no path left to explore. Otherwise, the decision on the level $d1$ must be flipped. This is not done automatically, CaDiCaL has no reason to flip it as the backtrack did not occur from a conflict but artificially from the user propagator.

There are two different options to continue the search from here: Either propagate the negation of the decision at level $d1 - 1$ via `cb_propagate` or decide the negation of the

decision at level `d1` via `cb_decide`. Both versions are implemented in CaDiCAll.

Continuing the search by propagating the negation of decision literals via the callback function `cb_propagate` appears to be a natural and elegant approach. When the solver propagates the negation of previously made decisions and subsequently backtracks, these literals no longer appear as decision literals and therefore do not need to be considered but simply can be backtracked over.

However, if a propagated literal participates in a conflict and is examined during conflict analysis, the solver requests a reason clause via `cb_add_reason_clause_lit`. The corresponding reason clause for such a literal consists of the negation of all decisions made at lower decision levels. Consequently, reason clauses do not need to be stored explicitly but can instead be generated on demand when required.

This approach has two main advantages compared to blocking clauses: First, reason clauses are typically smaller, as they only include decisions from lower levels of the search tree. Second, they are only constructed when needed, avoiding unnecessary extensions of the CNF.

When the search is continued by propagating the negation of the last decision, the last two of the four cases described above are handled as follows: Let `d` denote the previous decision.

- **The solver propagates only literals different from `d`, or performs no propagation at all.** In this situation, when `cb_propagate` is called, the literal $\neg d$ is propagated.
- **The solver propagates $\neg d$.** In this case, CaDiCAll still attempts to enforce $\neg d$ via `cb_propagate`. However, CaDiCaL ignores this request, as the literal is already assigned, and continues the search process.

While this approach optimizes blocking clauses, it still uses them. Consequently, CaDiCAll focuses on the second approach, which does not require adding any clauses at all. Nevertheless, the first approach remains available in CaDiCAll by running it via the `--reason (-r)` option and is also included in the benchmark evaluation presented in Section 6.

Continuing the search by deciding the negation of the previous decision via `cb_decide` does not require the addition of any clauses. After identifying the highest decision level `d1` with a decision count of one, CaDiCAll backtracks to decision level `d1 - 1`. It then enforces the negation of the previous decision by returning it in `cb_decide` and continues the search from there.

Recall that before `cb_decide` is invoked and the negation can be enforced, CaDiCaL propagates, which may influence the solver state. Let `d` denote the previous decision. In this context, the last two cases look like the following:

- **The solver propagates only literals different from d , or performs no propagation at all.** In this case, when `cb_decide` is invoked, CaDiCaL detects that a decision has already been made at the current decision level as its decision count is already one. Therefore, it enforces its negation by returning it.
- **The solver propagates $\neg d$ directly.** In this situation, the user propagator does not need to explicitly enforce this decision. Instead, it can allow the solver to select the next decision freely by returning zero in `cb_decide`.

Here it is essential to note that now decision can have a decision counter of two. This is not a problem when a model is found and the user propagator forces the solver to backtrack, since the decision level to backtrack to is explicitly determined by the user propagator. However, this situation must be handled more carefully when a conflict occurs and CaDiCaL performs backtracking independently. In this case, the decision counter must be taken into account to ensure that CaDiCaL backtracks to the correct decision level.

4.2.2 Conflict Induced Backtracking

When CaDiCaL encounters a conflict and performs backtracking, it is handled completely by CaDiCaL internally. Although CaDiCaL informs the user propagator via `notify_backtrack`, it does not allow the user propagator to interfere with the backtracking procedure.

With chronological backtracking activated, CaDiCaL backtracks to the most recent decision. Thus, it is guaranteed that CaDiCaL always backtracks exactly one decision level after a conflict occurred. However, when the search has been continued by deciding the negation of the last decision and not through propagating it. The last decision has a decision count of two. Thus, we do not want to backtrack to this decision level but instead backtrack to the most recent decision level with a decision count of one. However, backtracking after a conflict occurred is handled internally by CaDiCaL which does not know about the user propagator's decision counts. Therefore, CaDiCaL always backtracks to the most recent decision and continues the search from there, no matter the decision count.

If CaDiCaL backtracks to a decision with a count of two, it rediscovers a previously explored portion of the search space. Consequently, any progress made from that point onward must eventually be undone. CaDiCaL detects this situation by observing that the decision count at the current decision level exceeds two or that the decision variable is assigned again, and therefore enforces an additional backtracking. However, this mechanism may introduce computational overhead, as the solver can spend time performing propagations before the user propagator is able to intervene. In the worst case, the solver may propagate a long sequence of literals or even rediscover a total model before the user propagator can intervene by forcing the solver to backtrack further.

It remains to handle the case where CaDiCaL backtracks to a decision with a count of one. Although CaDiCaL derives a learned clause from the encountered conflict, it does not necessarily learn the negation of the old decisions. As a result, the same four cases may occur. While the first two remain unchanged, the latter two need to adapt:

- **The solver propagates the old decision d .** Thus, the solver starts to rediscover the old path. This is again detected directly in `notify_assignment` but can be handled the earliest when either `cb_decide` or `cb_check_found_model` is called. There, the solver is forced to backtrack further.
- **The solver encounters a conflict during propagating.** In this case a conflict-induced backtracking occurs again.
- **The solver only propagates variables that are different to decision d or none at all.** In that case the user propagator can continue normally and return the flipped decision in `cd_decide` or propagate the flipped decision with `cb_propagate`.
- **The solver propagates $\neg d$ itself.** Then the user propagator does not need to decide or propagate it manually and can let the next decision be chosen by the solver itself by returning zero in the next call to `cb_decide`.

4.3 Implicant Shrinking

Models (also called implicants) found by the solver are total. Models are total if they contain every variable. For a CNF $(x_1 \vee x_2) \wedge (x_1 \vee \neg x_2)$ a model enumerator returns two total models: $x_1 \wedge x_2$ and $x_1 \wedge \neg x_2$. As x_1 occurs in both models positively, the only difference lays in x_2 which occurs once positively and once negatively. In other words, if x_1 is positiv, it does not matter what the value of x_2 is, it is always a model for the CNF. Thus, these models can be merged into one model by dropping x_2 , resulting in one partial model x_1 . This reduces both, the amount of models in total and the size of models. It also may fasten up the solving itself. Normally, after finding the first model $x_1 \wedge x_2$ a model enumerator backtracks to the last decision that can be flipped. Assuming the decisions where x_1 at decision level one and x_2 at decision level two, a model enumerator would backtrack to decision level one and then continue with $\neg x_2$. However, if the model enumerator shrinks the model $x_1 \wedge x_2$ to x_1 the last decision that can be flipped is x_1 . Thus, the model enumerator backtracks to decision level zero and can continue with $\neg x_1$, skipping the branch with $\neg x_2$ completely. As the branch containing $\neg x_2$ could be arbitrary large, this may reduce the search space significantly.

Finding out which literals can be dropped is non-trivial. As described by Spallitta et al. [15], implicant shrinking algorithms from non-chronological backtracking based model enumerators do not guarantee mutual exclusiveness between different assignments. Thus, they introduce their own shrinking algorithm that is both: Compatible with chronological backtracking and that tries to cut the highest amount of literals while still ensuring mutual

exclusivity. This algorithm starts with the last literal on the trail and tries to drop as many literals as possible. A literal ℓ can be dropped if ℓ is a decision and all clauses in which ℓ occurs contain another literal ℓ' that satisfies the clause.

```
int implicant_shrinking(vector<int> stack) {
    int b = 0;
    int index = stack.size() - 1;
    while (index >= 0) {
        int v = stack[index];
        if (!decision(v)) {
            b = std::max(b, decision_level(v));
        } else if (decision_level(v) > b) {
            b = check_literal(v, b, stack, index);
        } else if (decision_level(v) == 0 || decision_level(v) == b) {
            break;
        }
        index--;
    }
    return b;
}
```

In CaDiCaL the two-watched-literal scheme is used. This allows the solver not to constantly check all clauses if the assignment of a literal changes. Instead, every clause of the CNF is only watched by two literals, let them be ℓ_1 and ℓ_2 . This is enough to track the status of the clause as long as the following invariant holds: At least one of the two watched literals is not falsified. If one watched literal ℓ_1 becomes falsified the solver tries to find a replacement in order to keep this invariant alive. If there is another literal ℓ' in that clause that is neither false nor ℓ_2 , it replaces ℓ_1 as the watching literal. When there exists no such literal ℓ' the solver knows that ℓ_2 is the only literal left that could satisfy the clause and forces ℓ_2 . If both literals ℓ_1 and ℓ_2 are falsified, the clause is falsified, yielding UNSAT.

If we want to drop a literal ℓ , it suffices to check all clauses that are watched by ℓ instead of all clauses in which ℓ occurs. If a model is found, we now at least one of the two watching literals for every clause is satisfied. Thus, dropping a literal that is not watched will never falsify a clause. However, if we drop a literal ℓ that is currently watching a clause c , we know there exists another literal $\ell' \in c$ that satisfies c . This literal ℓ' must now become the new watching literal for c . Otherwise, when trying to drop ℓ' , c would not be checked.

Going through all literals in all clauses watched by ℓ may introduce a bottleneck. Thus, Spallitta et al. [15] introduced a faster but more conservative algorithm exploiting the two-watched-literal scheme. Conservative meaning it may not drop all literals that could be dropped:

```

int check_literal(int v, int b, vector<int> stack, int index) {
    for (int lit : other_watching_literals(v)) {
        if (!satisfied_by(lit, get_n(stack, index))) {
            b = std::max(b, decision_level(v));
        }
    }
    return b;
}

```

Instead of checking all other literals in the watched clause, simply check the other literal ℓ' that watches that clause. If it satisfies the clause as well, the literal ℓ can be dropped. As ℓ' already watches the clause, nothing must be updated. This is more conservative as it may not drop literals that could be dropped. If ℓ' is falsified, but another literal in that clause is satisfied, ℓ still could be dropped but it is not. However, as this conservative approach does not need to traverse the clause and check each literal, it is significantly faster.

As IPASIR-UP does not allow any access to watches, this is implemented by using a backdoor to CaDiCaL directly. Section 5 discussed what is needed and how it could be added to the IPASIR-UP interface.

Adapting this implicant shrinking algorithm to CaDiCaL using IPASIR-UP may introduce a fault. As stated in Section 4.2 there are two different ways to flip a decision. If the decision is flipped by deciding its negation, the `implicant_shrinking` function must be adapted.

Consider the CNF $(x_1 \vee x_2 \vee x_3)$. Let x_2 and x_3 be the watched literals at the start and the decision order be $x_1 \rightarrow x_2 \rightarrow x_3$. The first model to be found will be $x_1 \wedge x_2 \wedge x_3$. Implicant shrinking will first remove x_3 as the other watched literal (x_2) still satisfies the clause. Then it will try to remove x_2 . However, as the other watched literal is x_3 which is no longer on the trail, it cannot continue. Thus, discovering the model $x_1 \wedge x_2$. Then the solver backtracks and flips x_2 by deciding its negation $\neg x_2$. Recall that $\neg x_2$ is still a decision. As x_2 is one of the watched literals but is now falsified, x_1 becomes the new watched literal, as it is the only literal in the clause that is neither false nor already watched. Then, the solver decides x_3 resulting in the model $x_1 \wedge \neg x_2 \wedge x_3$. Now, implicant shrinking starts again. First removing x_3 as the other watched literal (x_1) still satisfies the clause. Next, it tries to remove $\neg x_2$ as it is on the second-highest decision level. As $\neg x_2$ does not occur in any clauses, it cannot be a watched literal. Thus, removing it does not falsify any clauses. Then, it tries to remove x_1 . However, as the other watched literal is x_3 which was already removed, x_1 cannot be removed. Hence, yielding the model x_1 .

The implicant shrinking algorithm yielded both models $x_1 \wedge x_2$ and x_1 . These models are not distinct, as setting x_2 in the later generates the former.

This can be fixed by allowing implicant shrinking only to remove literals with a decision count of one. Consider a variable with decision count two to be checked by implicant

shrinking. There are two possibilities: First, the variable could not be removed when its decision count was one. Removing it now would trigger exactly a case like in the example above. Second, the variable was removed when its decision count was one. In that case the model enumerator would have backtracked over the variable and thus, resetting the decision count, which makes this case impossible to reach. Resulting in the following fix for the implicant shrinking algorithm:

```
int implicant_shrinking(vector<int> stack) {

    int b = 0;
    int index = stack.size() - 1;
    while (index >= 0) {
        int v = stack[index];
        if (!decision(v) || (decision(v) && decision_count(v) == 2)) {
            b = std::max(b, decision_level(v));
        } else if (decision_level(v) > b) {
            b = check_literal(v, b, stack, index);
        } else if (decision_level(v) == 0 || decision_level(v) == b) {
            break;
        }
        index--;
    }

    return b;
}
```

Flipping decisions by propagating their negations (Section 4.2) is not affected by this fix, as there the highest decision count is always one.

4.4 Deciding

The order in which decisions are made can heavily impact the performance of SAT solvers. The right decision can lead the solver into early conflicts, pruning large parts of the search space completely or skipping long nested parts. While in model enumeration the later benefit is lost as all models must be found, it still profits from early conflicts.

In order to search systematically through the whole search tree, a fixed variable order is the easiest. On each `cb_decide` call, it decides the next undecided variable positively. If the solver needs to backtrack, simply find the highest decision level with a positive decision. CaDiCAll was first implemented with a fixed variable order as it suffices for model enumeration. However, this constraint can be weakened and therefore, fasten up the performance while still preserving a systematic order.

First, the phase of a decided variable can be chosen freely, while keeping backtracking still simple enough. For each decision level a decision count is saved. When CaDiCAll

found a model and then determines the decision level it backtracks to, instead of looking for the highest decision level with a positive decision, it looks for the highest decision level with a decision count of one. A decision count of one means it has been decided exactly once.

Second, the order in which variables are decided can also be chosen completely freely, as long as after a backtracking the decision is just flipped. This enables the solver to profit from dynamic decision heuristics such as Variable State Independent Decaying Sum (VISIDS) which is known to improve solver performance significantly and thus, is a commonly used decision heuristic in modern SAT solvers. The performance difference between fixed and dynamic decision orders can be seen in Section 6.

The decision count is updated if a decision is made or flipped and reset if the decision is propagated or backtracked over. If the solver asks for a decision and the decision count on the current level is one. The decision must be flipped and therefore, `cb_decide` returns the negation of the old decision. Otherwise, `cb_decide` returns zero and thus, lets the solver decide itself.

4.5 Options

CaDiCaL allows enabling chronological backtracking. This option enforces the solver to always backtrack exactly one decision level. As described in Section 4.2 CaDiCaL relies on a systematic exploration of the search space and thus, CaDiCaL enables this option.

```
solver->set("chronoalways", true);
```

Furthermore, CaDiCaL provides a range of configurable features that are primarily designed to optimize solving performance. Some of these features however, are not compatible with a systematic exploration of the search space as they may reshape the search space or cause the solver to backtrack. Consequently, CaDiCaL explicitly disables these features:

- **Restarts**

Restarts are a widely used technique by SAT solvers to escape from stagnation by reinitializing the search while retaining learned clauses. Although effective in normal SAT solving, they disrupt the continuity of the search process and may cause the solver to revisit previously explored regions and thus, may lead the solver to rediscover models. Therefore, restarts are disabled.

```
solver->set("restarts", false);
```

- **Inprocessing**

Inprocessing techniques, such as clause elimination and subsumption, implicitly restart the solver. As restarts may cause the solver to rediscover already found models, inprocessing is turned off completely.

```
solver->set("inprocessing", false);
```

5 Extending IPASIR-UP

The core idea underlying CaDiCaL is to use IPASIR-UP as the sole interface to the underlying SAT solver. Consequently, all functionality required for model enumeration must be realized exclusively through this interface. This design choice offers a clear advantage: CaDiCaL can be readily adapted to any solver that supports IPASIR-UP without requiring modifications to the solver itself. However, IPASIR-UP in its current form does not provide all the functionality necessary for (efficient) model enumeration.

We propose three categories of extensions to IPASIR-UP. First, we introduce the features necessary to enable model enumeration through the IPASIR-UP interface. Second, additional features that allow implicant shrinking and third, we point out where duplication due to IPASIR-UP occurs and how to reduce it and thereby improve the overall performance of CaDiCaL.

5.1 Necessary Extensions for Model Enumeration

Most changes only affect the user propagator and not the solver directly. However, there are some features of CaDiCaL that break the chronological order of the search space and must be taken into account.

5.1.1 Backtracking After a Model Is Found

In conventional SAT solving, backtracking only occurs when the solver encounters a conflict. To continue the search, the solver must undo at least one previous decision and explore a different branch of the search space.

A similar situation arises when enumerating all satisfying assignments (models). Once the solver finds a model, the current assignment satisfies all clauses and therefore no conflict is present to trigger backtracking. Nevertheless, the search must continue in order to determine whether additional models exist. Thus, the solver must be forced to leave the current branch of the search space and explore alternative assignments.

When a model is found, `cb_check_found_model` is called. Here, IPASIR-UP allows the user propagator to observe and reject the found model by returning `false`. It then calls `cb_has_external_clause`. If the user propagator returns `true`, the solver repeatedly calls `cb_add_external_clause_lit` allowing the user propagator to add the negation of the model as a blocking clause. This forces the solver to backtrack and explore other branches of the search space. However, as we do not want to add blocking clauses, CaDiCaL always returns `false` in `cb_check_found_model`. This means, the solver is not forced to backtrack and would report the same model directly again, resulting in an infinite loop where the solver is stuck.

In order to allow the user propagator to maintain additional constraints, IPASIR-UP provides the function `force_backtrack`. This function is allowed to be called inside `cb_decide`. It enables the user propagator to force the solver to backtrack to a spe-

cific decision level before making a new decision. Forcing the solver to backtrack inside `cb_decide` does not introduce bad behavior as at this point, the trail is fully consistent: All assignments up to the current decision level do not falsify the CNF, they are not in conflict with any other propagations and no propagations are pending.

When `cb_check_found_model` is called, the trail is in an equally consistent state as in `cb_decide`: The trail fully satisfies the CNF and as it is a total assignment, there are no pending propagations. Thus, allowing `force_backtrack` in `cb_check_found_model` would not introduce any bad behavior.

To allow `force_backtrack` to be called from within `cb_check_found_model` simply the flag `forced_backt_allowed` must be set to `true` in the solver before calling `cb_check_found_model`.

```
forced_backt_allowed = true;
bool is_consistent = external->propagator->cb_check_found_model (etrail);
forced_backt_allowed = false;
```

This feature was added to the newest version of CaDiCaL's IPASIR-UP interface by Katalin Fazekas. Thus, allowing CaDiCaL to backtrack to the last decision after a model is found and continuing the search from there without adding blocking clauses.

5.1.2 Maintaining Chronological Backtracking

When the solver is about to make the next decision, it internally calls `ask_decision`, which then calls `cb_decide`. The user propagator is allowed to call `force_backtrack` inside `cb_decide`. Backtracking might make the decision returned by the user propagator invalid, as it might be already assigned by propagations due to the backtracking. However, instead of asking the user propagator again for the next decision, the solver makes the decision on its own. This imposes a problem as CaDiCaL does not always allow decision to be made by the solver, some decisions may be already fixed in order to force the solver to explore the search space chronologically. It can be fixed by calling `ask_decision` recursively instead of letting the solver decide. This way the user propagator can make a new decision after backtracking while keeping the exploration of the search space chronological.

An important note is that if the user propagator backtracks below the pseudo decision level of the solver, the solver goes back to the CDCL loop instead of making a decision. In that case the solver ignores the decision made by the user propagator, continues with the CDCL loop and calls `ask_decision` again when it is about to make the next decision. However, the CDCL loop may propagate literals that lead to paths that have already been explored. Thus, the user propagator must check whether the work done by the CDCL loop has led to such a path and if so, backtrack further. If not, the user propagator can either return the original decision or make a new one if the CDCL loop already propagated it.

```

if ((size_t) level < assumptions.size ()
    ((size_t) level == assumptions.size () && constraint.size ())) {
    return 0;
} else {
    return ask_decision();
};

```

5.1.3 Out Of Order Propagations

CaDiCaL tries to remove falsified literals in order to shrink the CNF and optimize the performance. Consequentially, this may produce new units. In order to keep blocking literals correctly, CaDiCaL propagates these units at the root level to derive all implied units. However, to propagate units at the root level, CaDiCaL potentially backtracks multiple levels, skipping branches that potentially lead to models. In order to avoid this, out-of-order propagation is deactivated.

```

bool Internal::propagate_out_of_order_units () {
    return true;
};

```

However, when CaDiCaL now tries to remove falsified literals it must not remove the literals which are out-of-order and are not fully propagated at the root level. Therefore, we filter for these literals in the garbage collection and propagation.

5.2 Implicant Shrinking

The implicant shrinking algorithm described in section 4.3 needs access to the watch-lists of a given literal in order to determine whether the other watched literal of a clause satisfies it or not. However, this is not supported via the IPASIR-UP interface. The implicant shrinking algorithm uses them read-only. Exposing a read-only reference to watch-lists is unproblematic and could be integrated into IPASIR-UP without any changes to CaDiCaL itself. Here it is important to note that literals can be named differently externally and internally. However, functions that translate from internal to external and vice versa already exist and thus, the internal representation easily could be completely hidden through the IPASIR-UP interface.

A function implementing this feature would look like the following: It takes an external variable as an input and returns all other literals that are currently watching the same clauses.

```

std::vector<int> other_watching_literals(int external_variable) {
    int internal_variable = internal->external->e2i[external_variable];
    int internal_literal = internal->external->vals[internal_variable] ?
    ⇨ internal_variable : -internal_variable;

    std::vector<int> other_watching_literals;
    for (CaDiCaL::Watch watch : internal->watches(internal_literal)) {
        int i_wl1 = watch.clause->literals[0];
        int i_wl2 = watch.clause->literals[1];
        int i_other_lit = internal_literal == i_wl1 ? i_wl2 : i_wl1;
        int e_other_lit = internal->externalize(i_other_lit);
        other_watching_literals.push_back(e_other_lit);
    }

    return other_watching_literals;
}

```

The user propagator can then check whether all other watching literals are satisfied by the remaining trail. If so, the literal can be dropped without falsifying any clause. Otherwise, the literal cannot be dropped as it is still needed to satisfy at least one clause.

If the sole purpose of this function is to be used by the implicant shrinking algorithm implemented from CaDiCaL, it could be optimized by checking if the other watching literal satisfies the clause instead of appending it to the list of other watching literals. This way, the function can return a boolean value indicating whether the literal can be dropped or not, without the need to return all other watching literals. This also allows early breaks, as the function can return false as soon as it finds a clause that is not satisfied by the other watching literal. However, the function would also need either access to the internal stack and the positions of literals, as the user propagator needs to check if the other watching literal is satisfied by the remaining trail, or it would need to be passed the current trail as an argument.

With this minor addition to the IPASIR-UP interface, the implicant shrinking algorithm can be implemented entirely through IPASIR-UP.

Currently, CaDiCaL implements this functionality by using this backdoor directly in the source code of CaDiCaL as it is not added to the IPASIR-UP interface yet. Thus, CaDiCaL implements the optimized version where early breaks are possible.

5.3 Optimization

The primary goal of this thesis was to investigate whether and how a model enumerator can be implemented using IPASIR-UP. Optimizing CaDiCaL and the IPASIR-UP interface to minimize runtime was not the main focus of this work. Nevertheless, while implementing CaDiCaL, we identified some bottlenecks that could be optimized by adapting

the IPASIR-UP interface.

5.3.1 Interacting With the Solver

When the solver calls the functions `notify_assignments` and `cb_check_found_models` it creates a vector containing assigned literals. When we timed CaDiCaL, in CNFs with many models, CaDiCaL spend around 40 % of its total time with vector operations. We found that when the solver is passing these vectors to the user propagator, the solver allocates new memory for each vector and frees it afterwards again. By making these vectors persistent, the total running time of CaDiCaL on CNFs with many models was cut in half. This optimization is implemented in CaDiCaL's version of the IPASIR-UP interface but is not yet part of the official IPASIR-UP interface.

5.3.2 Duplicating Datastructures

The most obvious performance bottleneck is the duplication of data structures.

In order to check where to backtrack to, if the solver rediscovers paths that have already been explored, or to check which literals can be dropped during implicant shrinking. CaDiCaL needs to maintain the values of variables, their decision levels and whether they are decisions or propagations. IPASIR-UP provides enough information to observe all the information through the notification callback functions `notify_assignment`, `notify_new_decision_level` and `notify_backtrack`. However, this means that every time a variable is assigned, it is not only stored internally by CaDiCaL but also externally by CaDiCaL. This does not only double the expenses but introduces an even bigger overhead as the information must also be passed from the solver to the user propagator through the callback functions. Thus, allowing read-only access to CaDiCaL's variable assignment and stack would make most external representation redundant. As this would be a bigger change to the IPASIR-UP interface and is not necessary to implement a model enumerator, it is not implemented in the current version of CaDiCaL. However, it could be added in the future to allow for more efficient model enumeration and other applications of IPASIR-UP as well.

5.3.3 Storing Models

Enumerating all models does not only require exponential time in the worst case but can also require exponential space as CNFs can have an exponential number of models. Thus, storing all models in memory may not be feasible for large CNFs. Hence, CaDiCaL provides the option to only count models without saving them. This can be enabled by the command line option `--count` or `-c`.

6 Experiments

The goal of the experimental evaluation is to assess the performance of CaDiCAll and determine whether implementing a model enumerator using IPASIR-UP is feasible. In particular, we aim to evaluate:

- The impact of implicant shrinking
- The effectiveness of reason-based propagation
- The influence of a fixed variable order
- The overall competitiveness compared to other model enumerators

6.1 Configuration

CaDiCAll supports multiple configurations, controlled via the command-line options:

- **Shrinking** (`--shrink` or `-s`)

Enables implicant shrinking using the algorithm described in the Section 4.3. This reduces models to smaller implicants, potentially finding models without actually computing them.

- **Reason-based propagation** (`--reason` or `-r`)

Instead of deciding the negation of decision literals, this mode propagates them using reason clauses. This can lead to faster backtracking as the solver can learn from the reason clauses and thus, may skip more of the search space. However, it also introduces an overhead as the solver needs to propagate the reason clauses and thus, may spend more time on propagations than on finding models.

- **Fixed variable order** (`--fixed` or `-f`)

Enabling this option forces the solver to explore the search space in a fixed order by always deciding the next undecided variable positively. However, this prevents the solver from using dynamic decision heuristics, which can lead to a worse performance.

All combinations of these options are evaluated, resulting in a total of eight configurations for CaDiCAll. Furthermore, CaDiCAll has the option to only count and not save the found models (`--count` or `-c`). All benchmarks are run with this option enabled, as saving models introduces a significant overhead in used space and IO-operations but does not impact the behavior of the solver itself.

6.2 Baseline

In order to properly assess CaDiCall’s performance, we compare it to **TabularAllSat** and **Dualiza**.

- **TabularAllSat**

As CaDiCall is a model enumerator implementing all state-of-the-art mechanisms defined and described by Spallitta et al.[15], the natural baseline for comparison is their solver: TabularAllSat. TabularAllSat is a CDCL based solver that uses reason-based propagations to flip variables and shrinks its models when found. Thus, running CaDiCall with shrinking (**-s**) and reason-based propagation (**-r**) enabled allows a one to one comparison to TabularAllSat. TabularAllSat also support counting models without actually storing them. Thus, TabularAllSat is also run with the option **-c** on the benchmarks, providing comparable performance stats.

- **Dualiza**

TabularAllSat and CaDiCall are both model enumerators that deliberately avoid the use of blocking clauses. Including Dualiza in the comparison therefore enables an explicit contrast with the classical enumeration approach that relies on blocking clauses. It additionally enables a more general assessment of the performance of TabularAllSat and CaDiCall, rather than restricting the evaluation to a purely pairwise comparison between the two approaches. Dualiza supports model counting (**-c**) as well. Thus, all three solvers are comparable.

6.3 Benchmark Sets

We compared the performance on the following three benchmark sets:

- **TabularAllSat Benchmarks**

A set of benchmarks specifically designed for evaluating AllSAT solvers, introduced by Spallitta et al. [15]. These benchmarks are crafted to have a high number of satisfying assignments and to allow for effective implicant shrinking.

- **SATLIB Benchmarks**

We also evaluate CaDiCall on a set of benchmarks from SATLIB [12]. These benchmarks are widely used in the SAT community and cover a variety of problem domains. As they are not specifically designed to benefit implicant shrinking, they may provide a different view on CaDiCalls performance. The benchmarks used from SATLIB include the planning instances: blocksworld (7 instances) and logistics (4 instances). All interval Series (4 instances) and SAT-encoded Quasigroup (or Latin square) instances (22 instances). This specific set is selected as they are known to have a high number of models and thus, are suitable for evaluating model

enumeration but are not especially designed for AllSAT and hence, not explicitly allow effective shrinking.

- **Model Counting Competition 2025 Benchmarks**

The benchmarks from the Model Counting Competition 2025 provide a complementary evaluation setting, as they originate from the domain of model counting rather than AllSAT solving. These instances are typically designed to be challenging with respect to counting the number of satisfying assignments, often featuring large and complex solution spaces.

In contrast to the TabularAllSat benchmarks, they are not constructed to facilitate implicant shrinking and thus, do not necessarily exhibit structural properties that can be exploited by shrinking techniques. Consequently, they serve as a means to evaluate the performance of CaDiCall under more general conditions, particularly in scenarios where the benefits of implicant shrinking are limited or absent.

6.4 Evaluation Metrics

All benchmarks were run on the Helix cluster of the university of Heidelberg with the following specifications: AMD EPYC Milan Processor kernels. Each benchmark was run with a time limit of 5000 seconds and a memory limit of 7000 megabyte. The benchmarks compare the time taken to find each model.

6.5 Results

The different benchmark sets yield notably divergent results. In the TabularAllSat benchmark suite (Figure 1), TabularAllSat clearly outperforms CaDiCall and performs better than Dualiza. In contrast, on the SATLIB benchmark set (Figure 2), TabularAllSat and all CaDiCall’s configurations without a fixed variable ordering exhibit comparable performance while Dualiza is unable to keep up. However, for the benchmark set derived from the Model Counting Competition 2025 (Figure 3), CaDiCall demonstrates a slight performance advantage to both, TabularAllSat and Dualiza.

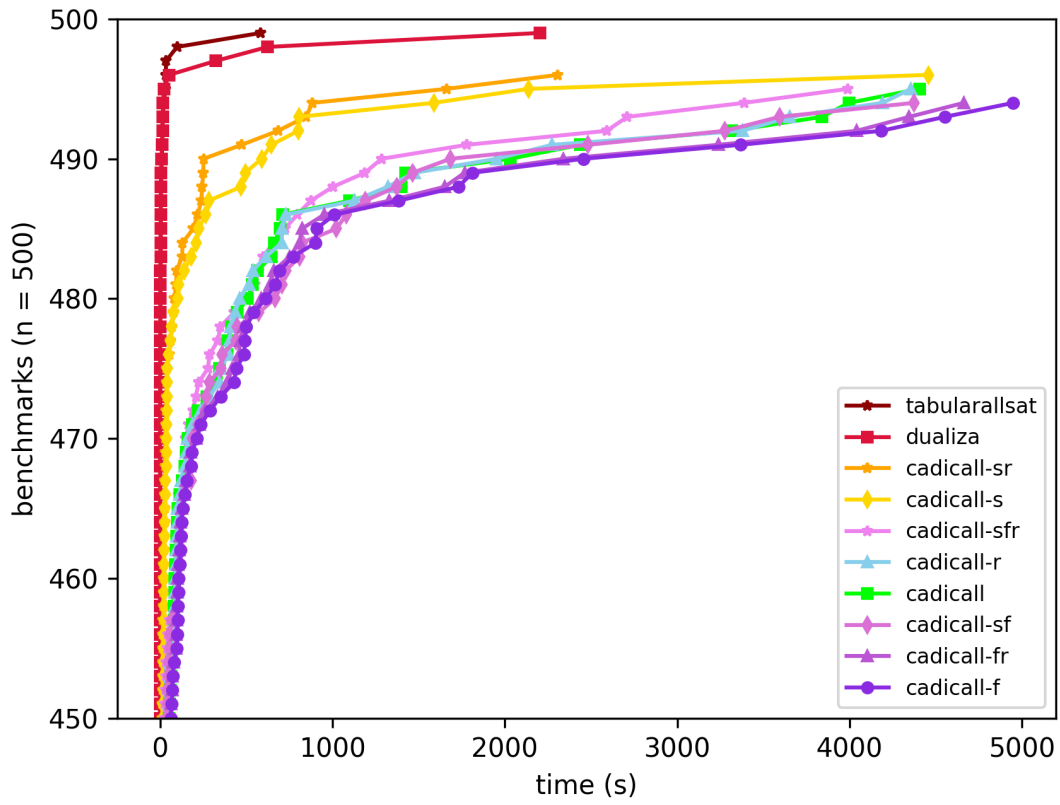


Figure 1: TabularAllSat Benchmarks.

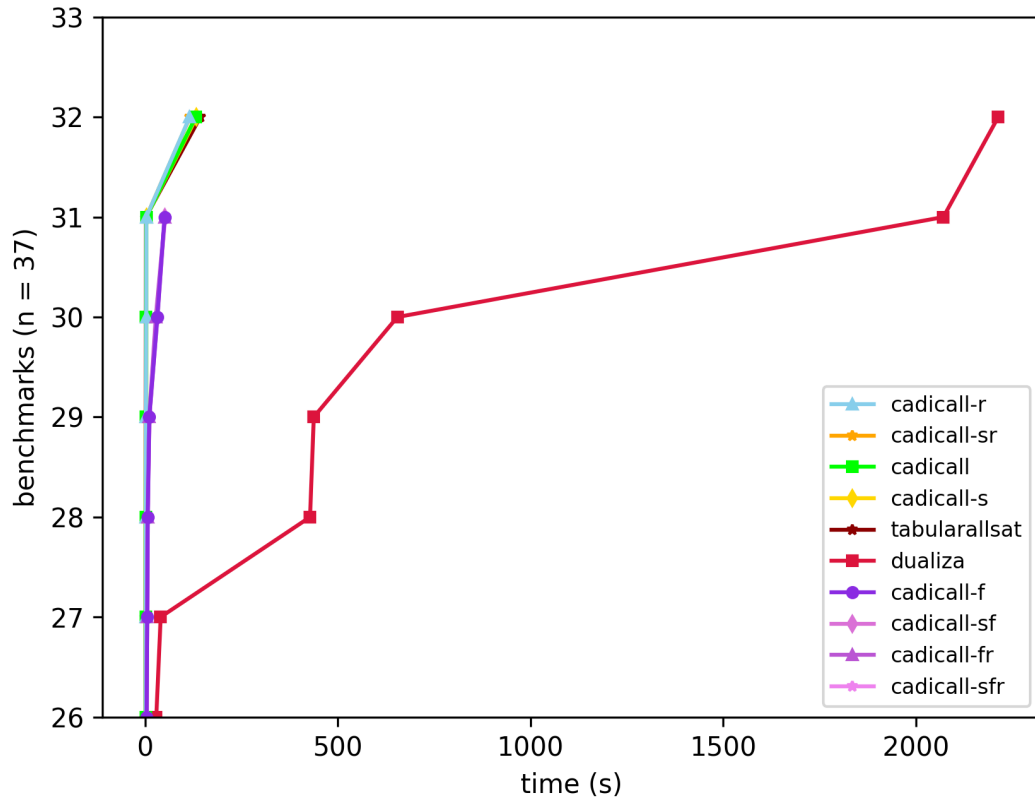


Figure 2: Custom Benchmarks.

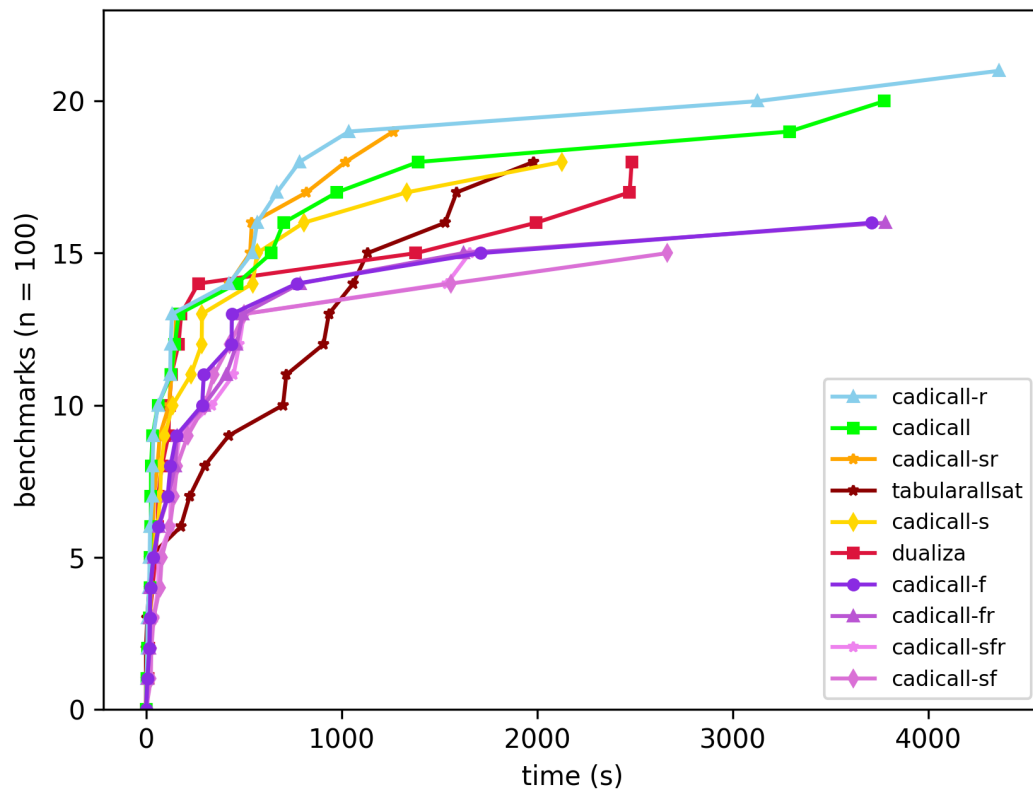


Figure 3: Model Counting Competition Benchmarks.

6.5.1 Benchmark Set from TabularAllSat

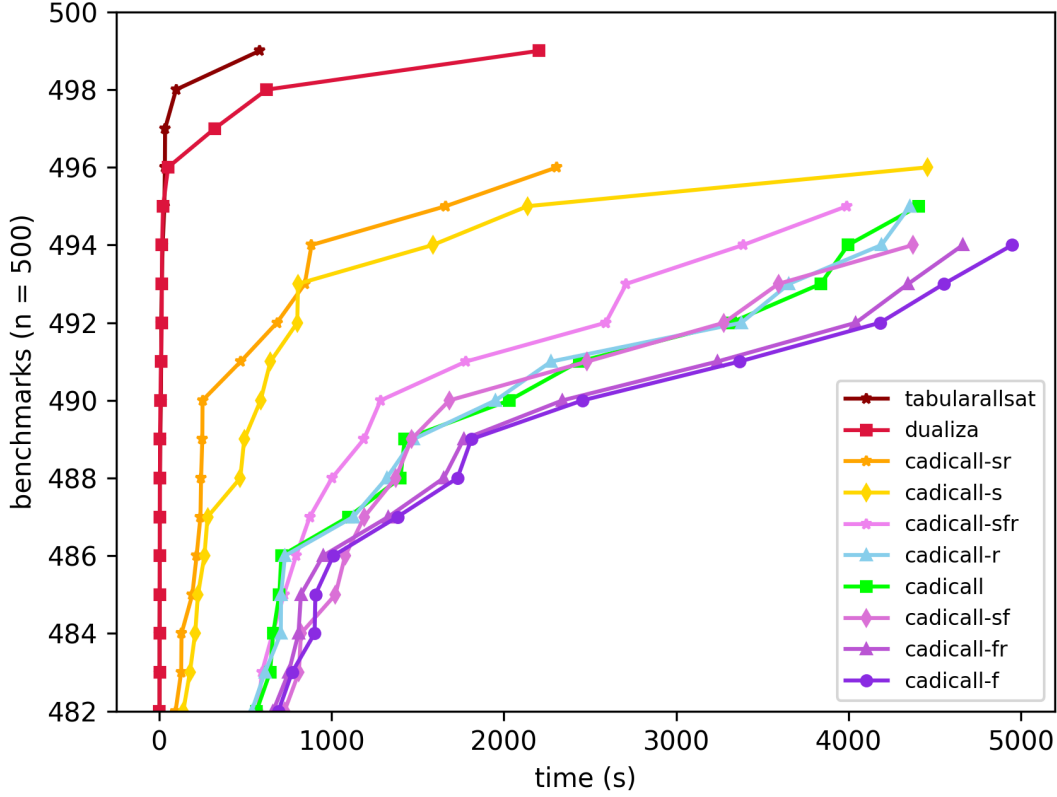


Figure 4: TabularAllSat vs. CaDiCAll Close Up.

A closer examination of the results for the TabularAllSat benchmark set (Figure 1) is shown in Figure 4. It reveals a clear performance advantage of TabularAllSat and Dualiza over CaDiCAll, both in terms of runtime and in the number of solved instances. More specifically, CaDiCAll configured with shrinking (`-s`), as well as with both shrinking and reason-based propagation (`-s -r`), exhibits performance comparable to TabularAllSat and Dualiza for approximately the first 480 instances. Beyond this point, however, the performance of CaDiCAll degrades significantly. Up to around 490 instances, CaDiCAll remains partially competitive with TabularAllSat and Dualiza. However, on the subsequent instances, its performance declines rapidly, ultimately failing to solve the final three benchmarks compared to TabularAllSat. These last three benchmarks (`BMS_k3_n100_m429_93`, `BMS_k3_n100_m429_30`, and `BMS_k3_n100_m429_303`) represent the most challenging instances in the benchmark set, with model counts of 19.414.570.229, 151.814.114.272, and 1.802.910.617.456, respectively.

Disabling shrinking results in a further deterioration of performance, which is consistent with expectations, as implicant shrinking is one of the most viable features and the benchmark set is explicitly designed to showcase effective implicant shrinking. Consequently, approaches that exploit this technique are inherently favored. Interestingly, when a fixed variable ordering is enforced (`-f`), shrinking no longer yields a performance

improvement for CaDiCAll. A plausible explanation for this observation is that, although the benchmarks are constructed to enable effective shrinking, this advantage depends on the solver’s ability to dynamically select a suitable variable ordering. When the variable ordering is fixed, the solver may be unable to identify an arrangement that facilitates effective shrinking, thereby reducing its overall impact on performance.

Overall, the configurations with a fixed variable order (`-f`) perform significantly worse than the best-performing CaDiCAll configuration as well as both, TabularAllSat and Dualiza. Nevertheless, for approximately the first 470 benchmark instances as shown in Figure 1, their performance remains comparable to that of the other CaDiCAll configurations, TabularAllSat and Dualiza. In addition, they exhibit behavior similar to those CaDiCAll configurations without a fixed variable order in which implicant shrinking is disabled.

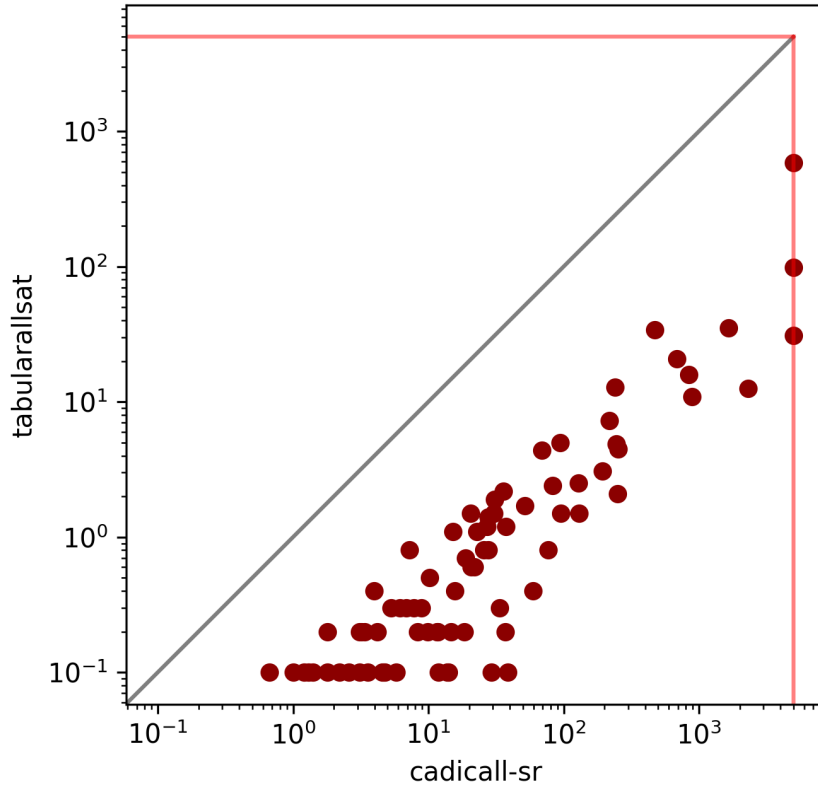


Figure 5: TabularAllSat vs. CaDiCAll.

A direct comparison between TabularAllSat and the best-performing configuration of CaDiCAll, with both shrinking and reason-based propagation enabled (Figure 5), reveals a performance difference of approximately one to two orders of magnitude.

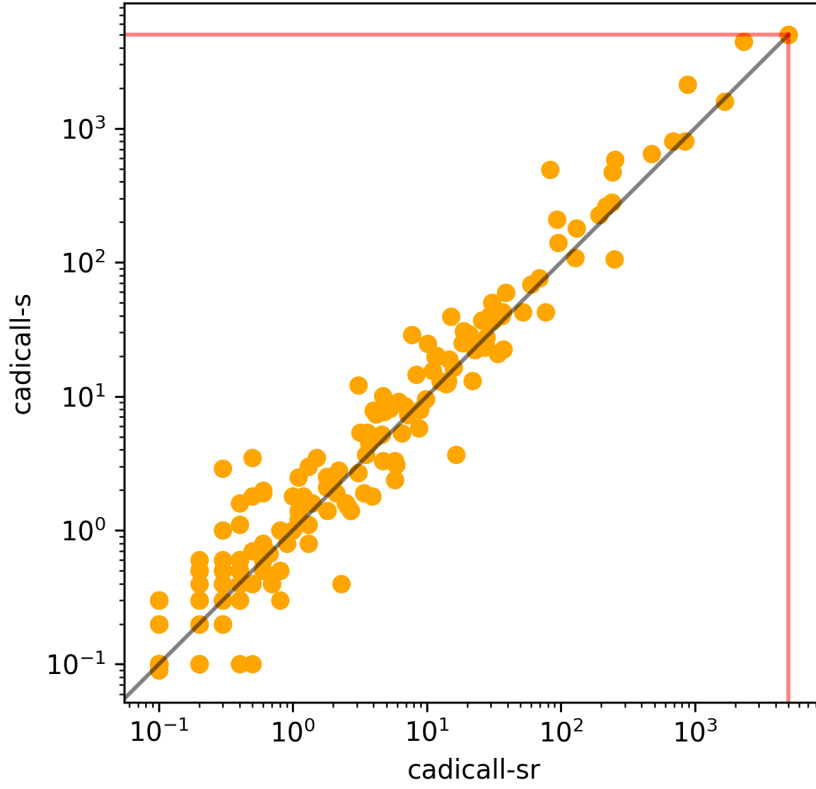


Figure 6: CaDiCAll sr vs s.

When comparing the performance of CaDiCAll with and without reason-based propagation, while shrinking is enabled (Figure 6), the overall results are largely similar with a slight advantage observed for configurations that employ reason-based propagation. One possible explanation is that, without reason-based propagation, CaDiCAll is more prone to revisiting previously explored regions of the search space, thereby incurring additional overhead in recovering from redundant explorations.

However, it is important to note that this effect may also be influenced by the design of the benchmark instances. As these benchmarks are specifically designed for Tabular-AllSat, which inherently utilizes reason-based propagation. They may implicitly favor configurations that incorporate this technique. Consequently, implicant shrinking may be more effective when reason-based propagation is enabled, as the benchmark structure is aligned with this operational paradigm.

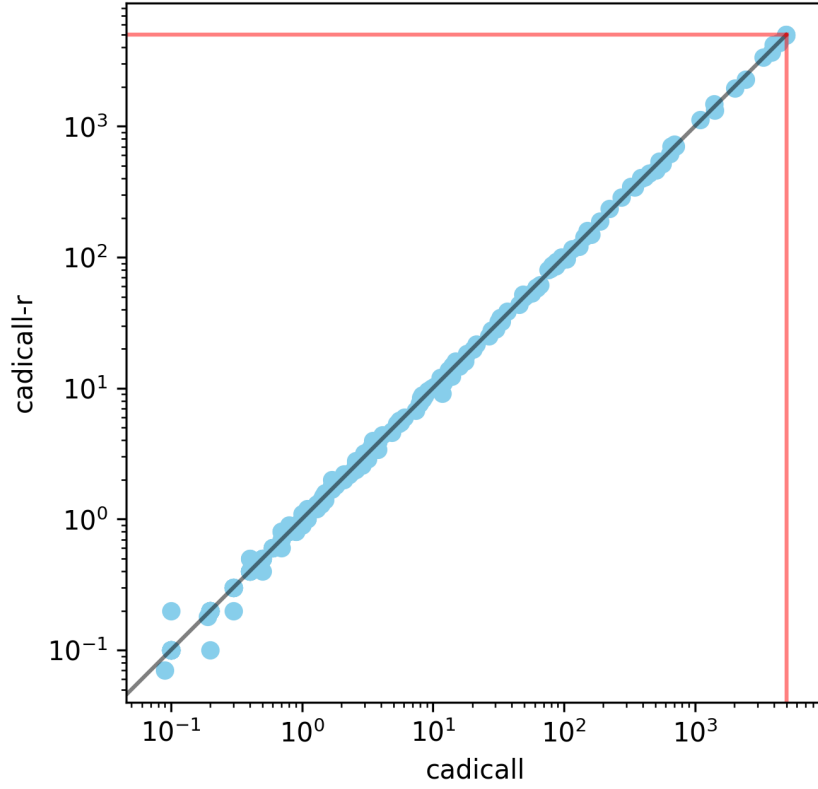


Figure 7: CaDiCAll vs CaDiCAll r.

This hypothesis is further supported by comparing the performance of CaDiCAll with and without reason-based propagation when shrinking is disabled (Figure 7). In this setting, the performance of both configurations is nearly identical. This suggests that reason-based propagation alone does not significantly influence performance when implicant shrinking is disabled. In turn, it suggests that either the primary benefit of reason-based propagation arises from its interaction with shrinking, or that the benchmark set inherently favors shrinking when reason-based propagations are employed.

6.5.2 Benchmark Set from SATLIB

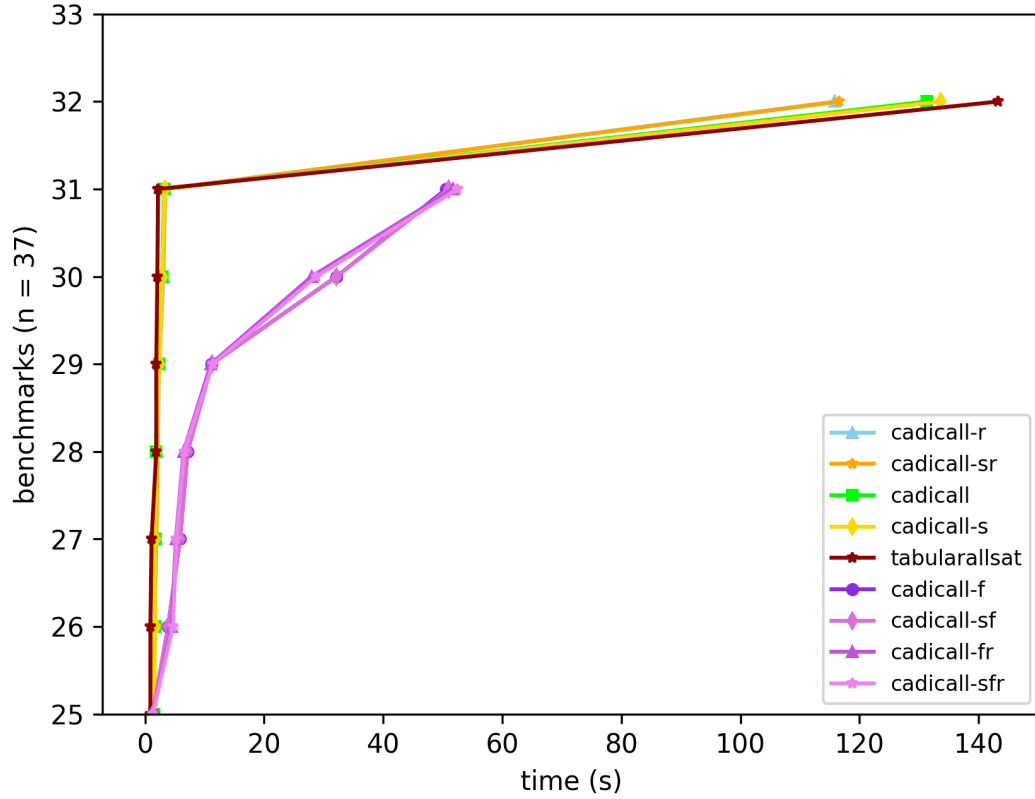


Figure 8: All Custom Benchmarks Close Up.

Figure 8 illustrated a closer look at the performance on the custom benchmark set from SATLIB that was shown in Figure 2 (Dualiza is excluded from this plot). CaDiCAll without a fixed variable order exhibits performance comparable to TabularAllSat on the custom benchmark set. Both approaches successfully solve 32 out of 37 instances within nearly identical runtimes (Figure 9). When enabling a fixed variable order, CaDiCAll solves 31 instances, one fewer than in the default configuration. However, for the instances that are solved, the runtime remains largely unchanged and is still comparable to both TabularAllSat and CaDiCAll without a fixed variable order. Furthermore, the results indicate that enabling or disabling implicant shrinking and reason-based propagation has no measurable impact on performance for this benchmark set. Specifically, configurations using shrinking (`-s`) and reason-based propagation (`-r`) perform identically to those that do not. This behavior is expected, as the custom benchmarks are not designed to benefit from shrinking, and therefore do not allow these techniques to effectively reduce the search space.

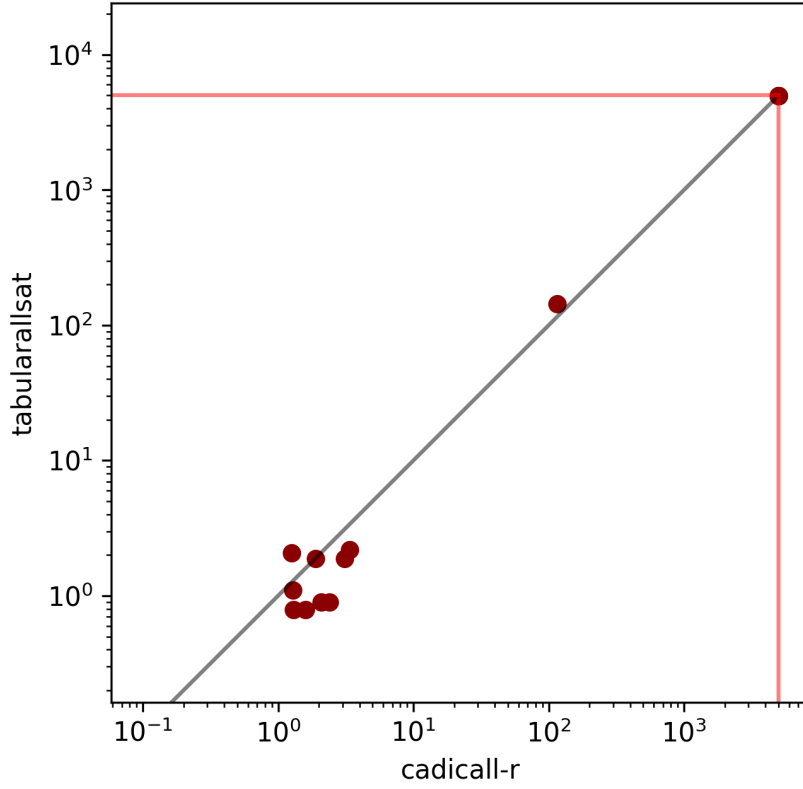


Figure 9: TabularAllSat vs CaDiCAll r.

Figure 9 shows a direct comparison between the performance of TabularAllSat and CaDiCAll with reason-based propagation on the custom benchmark set from SATLIB.

6.5.3 Benchmark Set from the Model Counting Competition 2025

The results of the benchmark set from SATLIB raises the question whether or not the TabularAllSat solver performs better on the TabularAllSat benchmarks because the instances favor it. To provide an additional angle, we conducted experiments on the benchmark set from the model counting competition from 2025 (Figure 3). A closer look is provided in Figure 10.

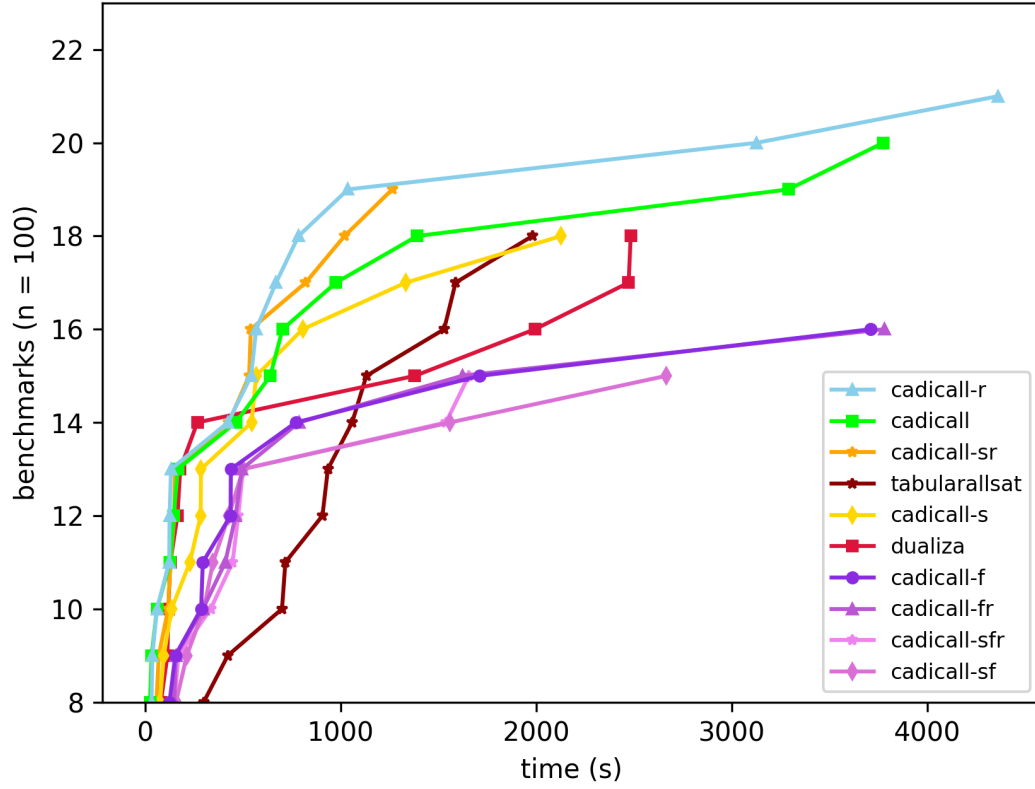


Figure 10: All Model Counting Competition Benchmarks from 2025 Close Up.

These results draw a different picture than what was shown by the TabularAllSat benchmarks. It was expected, that CaDiCall's performance may be closer to TabularAllSat's, as these benchmarks were not explicitly designed for TabularAllSat and implicant shrinking. However, it was not expected, that CaDiCall would perform better than TabularAllSat as CaDiCall is not optimized. An interesting note is that CaDiCall with reason-based propagation (21 solved instances) is not only faster but also able to solve more instances than TabularAllSat (18 solved instances). However, if we compare TabularAllSat which enables implicant shrinking to CaDiCall's configuration where implicant shrinking is activated, the difference in solved instances is not significant (CaDiCall with 19 vs TabularAllSat with 18 solved instances).

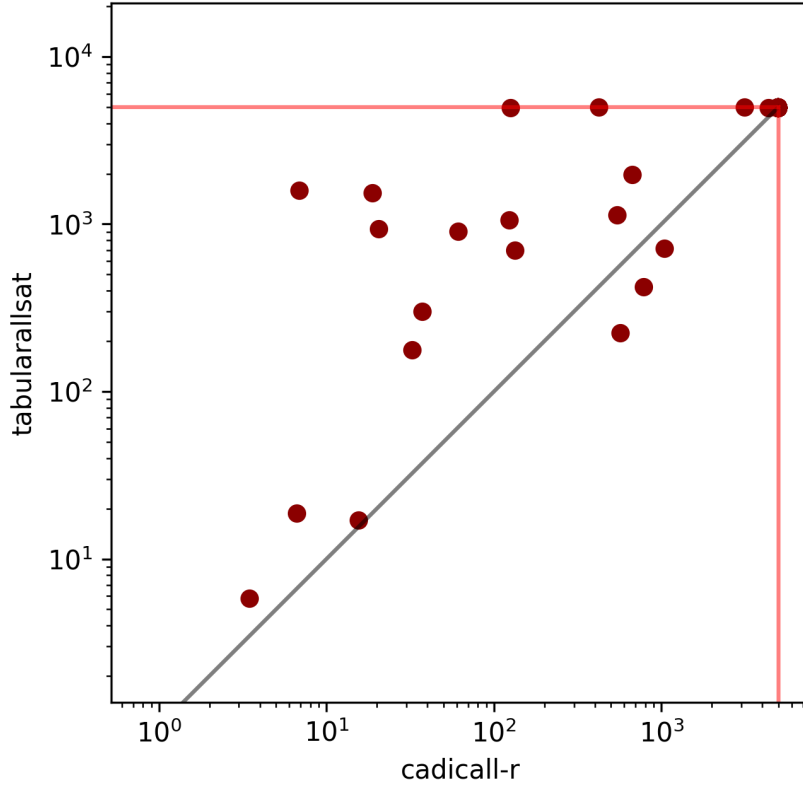


Figure 11: TabularAllSat vs CaDiCAll r.

Figure 11 presents a direct comparison of the performance of TabularAllSat and the best-performing configuration of CaDiCAll, namely the variant that employs only reason-based propagation.

Furthermore, Figure 10 reveals an additional noteworthy observation: CaDiCAll without reason-based propagation performs comparably to its counterpart with reason-based propagation on the first 16 benchmark instances. However, on the subsequent five instances it exhibits slightly inferior performance and ultimately solves one instance less.

6.6 Mismatching Results

While developing CaDiCAll, it was continuously tested against TabularAllSat and Dualiza with fuzzed CNFs from cnfuzz. In total all three solvers were compared on over 200 000 instances. However, these fuzzed CNFs were created with either the flag `--tiny` or `--small` in order to be manageable.

However, CaDiCAll, TabularAllSat and Dualiza do not yield the same model count for all instances of the three benchmark sets. In particular, there is one benchmark from the TabularAllSat benchmark set (BMS_k3_n100_m429_454) where TabularAllSat and CaDiCAll yield a model count of 16311020912 and Dualiza one of 7721086320. The custom benchmark set also includes one benchmark with a mismatch (qg4-09) and the model counting competition benchmark set includes four (mc2025_track1_79, ...93, ...95

and ... 101). On these five instances CaDiCAll yields the same count as Dualiza while TabularAllSat yields a different one.

6.7 Benchmark Results Overview

The experimental results demonstrate that the performance of CaDiCAll is highly dependent on both the chosen configuration and the structural properties of the benchmark instances.

The results on the TabularAllSat benchmark set reveal clear performance differences. Here, TabularAllSat and Dualiza consistently outperforms CaDiCAll, particularly on the most challenging instances. While CaDiCAll with implicant shrinking and especially in combination with reason-based propagation remains competitive on a large portion of the benchmarks, its performance degrades significantly on the hardest instances. This highlights the importance of shrinking for efficient model enumeration in instances with large solution spaces.

However, on the custom benchmark set, CaDiCAll performs comparably to TabularAllSat and better than Dualiza. Solving nearly the same number of instances as TabularAllSat within similar runtimes. In this setting, neither implicant shrinking nor reason-based propagation has a measurable impact, which is consistent with the fact that these benchmarks are not designed to benefit from shrinking.

The results of the custom benchmark set motivates further investigation. The model counting competition benchmark set showcase that CaDiCAll’s performance is competitive with TabularAllSat’s and Dualiza’s.

Furthermore, the experimental results show that reason-based propagation alone does not significantly improve performance in the absence of shrinking. Its benefits become apparent primarily when combined with shrinking, suggesting a strong interaction between these two techniques. This indicates that reason-based propagation is most effective when it supports the reduction of implicants rather than acting as an independent optimization.

Another key observation concerns the role of variable ordering: Enforcing a fixed variable order consistently degrades performance across the TabularAllSat benchmarks and largely diminishes the benefits of implicant shrinking. This suggests that dynamic variable ordering is crucial for enabling effective shrinking, as it allows the solver to adapt to the structure of the instance. In contrast, the impact of a fixed variable order is less pronounced on the custom benchmarks, further emphasizing that the effectiveness of these techniques is closely tied to the characteristics of the problem instances.

Overall, the results indicate that integrating model enumeration into the solver via IPASIR-UP is a viable approach. However, achieving competitive performance requires careful exploitation of shrinking and solver heuristics. In particular, the combination of implicant shrinking, reason-based propagation, and dynamic variable ordering emerges as the most effective configuration for challenging benchmark instances.

7 Future Work

CaDiCAll demonstrates that implementing a model enumerator without blocking clauses via the IPASIR-UP interface is feasible. However, achieving this requires certain extensions to both the IPASIR-UP interface and the underlying solver, CaDiCaL.

Future work could focus on extending the IPASIR-UP interface to provide read-only access to internal solver structures, such as the watch lists, and on optimizing the communication between the solver and the user propagator. Additionally, performance could be improved by reducing the duplication of data structures, for instance by enabling controlled access to solver-internal data.

More substantial improvements may be achieved through modifications to CaDiCaL itself. In particular, enabling a strictly chronological exploration of the search space requires the ability to fully disable solver features that interfere with this process, including out-of-order propagations. Providing such functionality as an optional configuration within CaDiCaL would allow the implementation of a model enumerator via IPASIR-UP without requiring further modifications to the solver.

Taken together, these extensions and adaptations have the potential to significantly improve the performance of CaDiCAll. This raises the question of whether, with these enhancements, CaDiCAll could become competitive with state-of-the-art model enumeration approaches such as TabularAllSat.

8 Conclusion

The goal of this thesis was to assess whether it is feasible to implement a model enumerator without blocking clauses using the IPASIR-UP interface. While the developed prototype, CaDiCaL, demonstrates that such an approach is conceptually possible, its implementation and the results reveal important practical limitations.

In particular, neither the current version of the IPASIR-UP interface nor the existing implementation of CaDiCaL fully support the requirements for a realization of this approach. On the one hand, the limitations of IPASIR-UP are not fundamental: As discussed in Section 5, the missing functionality of the IPASIR-UP interface such as allowing to backtrack after a model is found or access to the watch lists of literals required for implicant shrinking and even the optimizations for improved communication between the solver core and the user propagator can be added with comparatively moderate effort. These extensions are well-scoped and do not fundamentally challenge the design philosophy of the interface.

On the other hand, more substantial obstacles arise from the internal behavior of CaDiCaL itself. In particular, features such as out-of-order propagations and other non-chronological aspects of modern CDCL solving interfere with the strictly chronological exploration of the search space that is required for efficient model enumeration without blocking clauses. Enabling such a traversal is therefore not merely a matter of interface design, but requires modifications to the solver’s internal mechanisms that may have non-trivial impact on the solver’s behavior. This elevates the question if such a model enumerator is feasible, to the question: Does the required effort to adapt the solver is justified by the potential performance gains and the benefits of modularity of using CaDiCaL as the underlying solver through the IPASIR-UP interface.

The experimental evaluation provides a differentiated answer to this question. On benchmark instances specifically designed to stress model enumeration techniques, such as implicant shrinking and reason-based propagation, CaDiCaL does not achieve the performance of specialized approaches like TabularAllSat or Dualiza. However, the performance degradation arises particularly on structurally complex instances with many models. In such cases, the communication between the user propagator and the solver core are especially relevant for performance as their overhead grows with a higher model count. Thus, optimizing the communication may reduce this performance difference.

Furthermore, the results on more general benchmark sets, including SATLIB and instances from the Model Counting Competition, paint a more nuanced picture. In these settings, CaDiCaL performs competitively and, in some cases, even surpasses TabularAllSat and Dualiza. This indicates that the practical viability of the approach is highly dependent on the structure of the input formulas and that blocking-clause-free enumeration via IPASIR-UP is not inherently inefficient.

Taken together, these findings suggest that the feasibility of this approach ultimately hinges on solver support for strict chronological search. If CaDiCaL was extended to allow

such behavior as an optional configuration, the remaining interface-level enhancements appear manageable. Under these conditions, using CaDiCaL as the core solver while combining it with advanced model enumeration techniques such as implicant shrinking could enable a competitive model enumerator without blocking clauses.

In conclusion, while the current state of IPASIR-UP and CaDiCaL does not yet allow for a fully efficient realization of this idea, the approach shows clear potential. Whether this potential justifies the required modifications to the solver remains an open question, but the results of this thesis indicate that it is a promising direction for future research.

References

- [1] Kiyoshi Akama and Ekawit Nantajeewarawat. Solving query-answering problems using all-solution satisfiability solvers: Algorithm and correctness. In *2013 Third World Congress on Information and Communication Technologies (WICT 2013)*, pages 79–84, 2013. doi: 10.1109/WICT.2013.7113113.
- [2] Gilles Audemard and Laurent Simon. On the Glucose SAT solver. *International Journal on Artificial Intelligence Tools (IJAIT)*, 27(1):1–25, 2018. URL <https://univ-artois.hal.science/hal-03299473>.
- [3] Tomáš Balyo, Armin Biere, Ashlin Iser, and Carsten Sinz. Sat race 2015. *Artificial Intelligence*, 241:45–65, 2016. ISSN 0004-3702. doi: <https://doi.org/10.1016/j.artint.2016.08.007>. URL <https://www.sciencedirect.com/science/article/pii/S0004370216300984>.
- [4] Armin Biere and W. Kunz. Sat and atpg: Boolean engines for formal hardware verification. pages 782–785, 12 2002. ISBN 0-7803-7607-2. doi: 10.1109/ICCAD.2002.1167620.
- [5] Armin Biere, Marijn Heule, Hans van Maaren, and Toby Walsh, editors. *Handbook of Satisfiability*, volume 336 of *Frontiers in Artificial Intelligence and Applications*. IOS Press, Amsterdam, 2 edition, 2021.
- [6] Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. Cadical 2.0. In Arie Gurfinkel and Vijay Ganesh, editors, *Computer Aided Verification*, pages 133–152, Cham, 2024. Springer Nature Switzerland. ISBN 978-3-031-65627-9.
- [7] Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. CaDiCaL, Gimsatul, IsaSAT and Kissat entering the SAT Competition 2024. In Marijn Heule, Markus Iser, Matti Järvisalo, and Martin Suda, editors, *Proc. of SAT Competition 2024 – Solver, Benchmark and Proof Checker Descriptions*, volume B-2024-1 of *Department of Computer Science Report Series B*, pages 8–10. University of Helsinki, 2024.
- [8] Marijn Heule Cayden Codel, Katalin Fazekas and Ashlin Iser. Sat competition 2025. <https://satcompetition.github.io/2025/index.html>, 2025. Accessed: 2026-05-08.
- [9] Stephen A. Cook. The complexity of theorem-proving procedures. *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pages 151–158, 1971.
- [10] Katalin Fazekas, Aina Niemetz, Mathias Preiner, Markus Kirchweger, Stefan Szeider, and Armin Biere. IPASIR-UP: User Propagators for CDCL. In Meena Mahajan and Friedrich Slivovsky, editors, *26th International Conference on Theory*

- and Applications of Satisfiability Testing (SAT 2023)*, volume 271 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 8:1–8:13, Dagstuhl, Germany, 2023. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. ISBN 978-3-95977-286-0. doi: 10.4230/LIPIcs.SAT.2023.8. URL <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.SAT.2023.8>.
- [11] Dror Fried, Alexander Nadel, and Yogev Shalmon. AllSAT for Combinational Circuits. In Meena Mahajan and Friedrich Slivovsky, editors, *26th International Conference on Theory and Applications of Satisfiability Testing (SAT 2023)*, volume 271 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 9:1–9:18, Dagstuhl, Germany, 2023. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. ISBN 978-3-95977-286-0. doi: 10.4230/LIPIcs.SAT.2023.9. URL <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.SAT.2023.9>.
- [12] Holger H. Hoos and Thomas Stützle. Satlib: An online resource for research on sat. In I. P. Gent, H. van Maaren, and T. Walsh, editors, *SAT 2000*, pages 283–292. IOS Press, 2000.
- [13] Leonid A. Levin. Universal search problems. *Problems of Information Transmission*, 9(3):265–266, 1973.
- [14] Harish Kumar Sahu, N. Rajesh Pillai, Indivar Gupta, and R. K. Sharma. Smt solver-based cryptanalysis of block ciphers. *SN Comput. Sci.*, 1(3), May 2020. doi: 10.1007/s42979-020-00181-4. URL <https://doi.org/10.1007/s42979-020-00181-4>.
- [15] Giuseppe Spallitta, Roberto Sebastiani, and Armin Biere. Disjoint partial enumeration without blocking clauses. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(8):8126–8135, Mar. 2024. doi: 10.1609/aaai.v38i8.28652. URL <https://ojs.aaai.org/index.php/AAAI/article/view/28652>.
- [16] Yakir Vizel, Georg Weissenbacher, and Sharad Malik. Boolean satisfiability solvers and their applications in model checking. *Proceedings of the IEEE*, 103(11):2021–2035, 2015. doi: 10.1109/JPROC.2015.2455034.